

Декоратор Markup

Это библиотека для работы с форматом [JsonML](#) на клиенте. Представлена контролем [Controls/markup:Decorator](#) и модулем [Controls/markup:Converter](#).

Контроль Controls/markup:Decorator

Контроль [Controls/markup:Decorator](#) наследуется от [UI/Base:Control](#) и служит для вставки вёрстки в формате [JsonML](#) в шаблон.

Пример:

```
<Controls.markup:Decorator value="{{ json }}" />
```

Таким образом, из JsonML:

```
[["p", "Hello world!"]]
```

Получится вёрстка:

```
<div><p>Hello world</p></div>
```

Внешний div добавляется всегда, чтобы гарантировать только один корневой тег.

Конвертер Controls/markup:Converter

Модуль [Controls/markup:Converter](#) служит для конвертации JsonML-массива в HTML-строку и обратно. Работает только на клиентской части. На сервере для такой конвертации существуют специальные методы бизнес-логики.

Конвертер экспортирует три метода:

1. [htmlToJson](#) — конвертирует HTML-строку в JsonML-массив. В атрибуты корневого тега добавляет версию JsonML.
2. [jsonToHtml](#) — конвертирует JsonML-массив в HTML-строку. Имеет общий механизм с контролем, поэтому тоже добавляет корневой **div**.
3. [deepCopyJson](#) — создаёт полную копию по значению JsonML-массива.

Преобразователи узлов

Иногда нужно не отображать JsonML-вёрстку, как она есть, а немного изменить часть узлов перед отображением. Для этого существует механизм преобразователей узлов, представленных опцией [tagResolver](#) контроля **Controls/markup:Decorator**, или вторым аргументом метода [jsonToHtml](#) модуля **Controls/markup:Converter**.

Преобразователю можно передать дополнительную информацию с помощью опции [resolverParams](#) контроля **Controls/markup:Decorator**:

```
1 <Controls.markup:Decorator
2   value="{{ json }}"
3   tagResolver="{{ tagResolver }}"
4   resolverParams="{{ resolverParams }}" />
```

Или с помощью третьего аргумента метода [jsonToHtml](#) модуля **Controls/markup:Converter**:

```
const html = Converter.jsonToHtml(json, tagResolver, resolverParams);
```

Общее API преобразователя узлов

Преобразователь представляет собой функцию, принимающую текущий узел JsonML-дерева (value), его родителя (parent) и параметры преобразователя (resolverParams). Функция должна вернуть уже окончательный вид этого узла - как он отобразится.

Маркёр декоратор обоих направлений сам вызывает функцию для каждого узла от родителя к потомкам, рекурсивно. Оригинальные узлы не должны модифицироваться преобразователем, он должен либо вернуть старый узел неизменным, либо создать и вернуть новый. Стоит учесть, что к потомкам нового узла преобразователь применяться уже не будет.

Пример преобразователя узлов, который будет оборачивать в "strong" все узлы с именем "span":

```
1 function myResolver(value, parent) {
2     // Нас интересует только узлы с именем "span".
3     if (!Array.isArray(value) || value[0] !== 'span') {
4         return value;
5     }
6
7     // Не будем трогать узлы, у которых родители - "strong".
8     if (parent && parent[0] === 'strong') {
9         return value;
10    }
11
12    // Возвращаем новый узел, равный старому, обернутому в "strong".
13    return ['strong', value];
14 }
```

Готовые платформенные преобразователи узлов

Далее рассмотрим готовые преобразователи, лежащие в библиотеке Markup декоратора.

Controls/markup:highlightResolver

Преобразователь используется, если нужно выделить цветом часть текста. Подстроку, которую нужно выделить, следует передать через [resolverParams](#).

```
1 // TypeScript
2 import { _highlightResolver } 'Controls/markup';
3 import { Control } from 'UI/Base';
4 import * as template from 'wml!Demo/HighlightResolverDemo';
5
6 export default class HighlightResolverDemo extends Control {
7     _template = template;
8     tagResolver = _highlightResolver;
9     resolverParams = {
10         textToHighlight: 'ell'
11     };
12     json = [[ 'p', 'Hello world!' ]];
13 }

```

```
1 <!-- WML -->
2 <Controls.markup:Decorator
3     value="{{ json }}"
4     tagResolver="{{ tagResolver }}"
5     resolverParams="{{ resolverParams }}" />
```

В итоге отрисовывается текст "Hello world!", в котором фрагмент "ell" выделится цветом.

Control/decorator:InnerText

Преобразователь используется, чтобы получить текстовое содержимое JsonML-массива.

```
1 // TypeScript
2 import('Controls/markup').then(({ Converter, InnerText }) => {
3   const json = [['p', 'first paragraph'], ['p', 'second paragraph'],
4     ['p', {class: 'myClass'}, 'Hello ', ['strong', 'my'], ' world!']];
5   const text = Converter.jsonToHtml(json, InnerText ); // 'first
    paragraph\nsecond paragraph\nHello my world!\n'
6 });
```

Controls/markup:noOuterTag

Преобразователь используется для того, чтобы jsonToHtml Converter не добавлял корневой **div**.

```
1 // TypeScript
2 import('Controls/markup').then(({ Converter, noOuterTag }) => {
3   const json = [['p', 'text']];
4   const htmlWithOuterTag = Converter.jsonToHtml(json); // '<div>
    <p>text</p></div>'
5   const htmlWithoutOuterTag = Converter.jsonToHtml(json, noOuterTag); //
    '<p>text</p>'
6 });
```

Controls/markup:linkWrapResolver

Преобразователь используется для оборачивания ссылок из текста в теги `<a>`.

```
1 // TypeScript
2 import { linkWrapResolver } 'Controls/markup';
3 import { Control } from 'UI/Base';
4 import * as template from 'wml!Demo/LinkWrapResolverDemo';
5
6 export default class LinkWrapResolverDemo extends Control {
7   _template = template;
8   tagResolver = linkWrapResolver;
9   json = [['p', 'Переходи по ссылке vk.com/id0' ]];
10 }

```

```
1 <!-- WML -->
2 <Controls.markup:Decorator
3   value="{{ json }}"
4   tagResolver="{{ tagResolver }}" />
```

В итоге получится вёрстка:

```
1 <div><p>Переходи по ссылке <a href="http://vk.com/id0">vk.com/id0</a>
  </p></div>
```

[deprecated] Controls/markup:linkDecorate



Механизм устарел!

Актуальную информацию читайте [здесь](#).

Преобразователь используется для преобразования обычных ссылок в декорированные, т.е. изображающиеся в виде картинки-плашки.

Сначала он находит ссылки в тексте, как в `linkWrapResolver`. Потом все ссылки, и которые нашлись в тексте, и которые уже были в теге `<a>`, преобразователь проверяет на требования декорированной ссылки. И, если по всем требованиям ссылка подходит, то заменяет обычную ссылку картинкой определённого вида с помощью сервиса декорирования ссылок.

```
1 // TypeScript
2 import { linkDecorate } 'Controls/markup';
3 import { Control } from 'UI/Base';
4 import * as template from 'wml!Demo/LinkDecorateResolverDemo';
5
6 export default class LinkDecorateResolverDemo extends Control {
7     _template = template;
8     tagResolver = linkDecorate;
9     json = [[ 'p', 'Переходи по ссылке ', ['a', {href:
10 'http://github.com'}], 'http://github.com' ] ]];
11 }
12
13 <!-- WML -->
14 <Controls.markup:Decorator
15     value="{{ json }}"
16     tagResolver="{{ tagResolver }}" />
```

В итоге, если в окружении работает сервис декорирования ссылок, результат будет выглядеть так:

